

Types of Operating Systems

1. Batch Operating System

A **Batch Operating System** processes a group (batch) of jobs together **without requiring user interaction** during execution. **Similar jobs are collected and executed** one after another **automatically**.

How does it work?

1. Users submit jobs to the operator.
2. Jobs with similar requirements are grouped into a batch.
3. The operating system loads and executes each job sequentially.
4. After one job finishes, the next job starts automatically.

Advantages

- **Reduces manual intervention.**
- **Improves CPU utilization** compared to serial processing.
- Suitable for large **repetitive tasks**.
- Efficient for processing **huge amounts of data**.

Limitations

- **No interaction between user and program** during execution.
- **Long turnaround time.**
- **Difficult to debug programs.**
- A **failure** in one job may affect subsequent jobs.

Real-World Examples

- Payroll processing in companies.
- Bank transaction processing.
- Utility bill generation.
- Examination result processing.

2. Time-Sharing Operating System

A **Time-Sharing Operating System** allows **multiple users to use a computer simultaneously** by sharing CPU time among them.

How does it work?

- The CPU divides its time into small intervals called **time slices**.
- Each user or process gets a small amount of CPU time.
- The CPU switches rapidly between users, creating the illusion that everyone has their own computer.

Advantages

- **Fast response** time.
- Supports **multiple users** simultaneously.
- **Efficient CPU** utilization.
- **Interactive** computing environment.
- **Easy program development** and **debugging**.

Limitations

- More complex than batch systems.
- **Security and data protection challenges**.
- **Performance** may **decrease** when **many users are active**.
- Requires **sophisticated scheduling** algorithms.

Applications

Application	Why Time-Sharing is Used
University computer labs	Many students access the same server simultaneously.
Banking systems	Multiple employees access customer databases at the same time.
Cloud servers	Thousands of users share server resources.
Web hosting servers	Many websites run on the same machine.
Office networks	Multiple users access shared files and printers.
Online reservation systems	Many customers interact with the system simultaneously.
Software development servers	Multiple developers use the same server.
Modern personal computers	Multiple applications run concurrently (browser, music player, editor, etc.).

- UNIX systems in universities.

- Linux servers.
- Modern cloud-based systems.
- Multi-user database systems.

3. Real-Time Operating System (RTOS)

A **Real-Time Operating System (RTOS)** is designed to provide **responses within a strict time limit**. The correctness of the system depends not only on the result but also on when the result is produced.

How does it work?

- Tasks are **assigned priorities**.
- **Critical tasks receive immediate CPU access.**
- The operating system **guarantees predictable response times**.

Types of RTOS

Hard Real-Time System

Missing a deadline is unacceptable.

Examples:

- Aircraft control systems
- Medical life-support systems
- Nuclear power plant controls

Soft Real-Time System

Occasional deadline misses are acceptable.

Examples:

- Video streaming
- Online gaming
- Multimedia applications

Feature	Hard Real-Time OS	Soft Real-Time OS
Definition	A system where every task must be completed before its deadline.	A system where deadlines are important but occasional misses are acceptable.
Deadline Requirement	Strict and mandatory.	Flexible and desirable.
Consequence of	System failure, safety risk,	Reduced performance or

Missing Deadline	or catastrophic consequences.	quality, but system continues to function.
Response Time	Guaranteed and predictable.	Fast but not always guaranteed.
Priority	Critical tasks always get highest priority.	Tasks are prioritized, but some delays can be tolerated.
Reliability Requirement	Extremely high.	Moderate to high.
Scheduling	Deterministic scheduling algorithms.	Best-effort scheduling.
Application Area	Safety-critical and mission-critical systems.	Multimedia and commercial applications.

Advantages

- **Predictable** and **reliable** performance.
- **Quick response** to critical events.
- **High** system **stability**.
- Suitable for **mission-critical applications**.

Limitations

- **Expensive** to design and maintain.
- **Limited flexibility**.
- **Complex scheduling** mechanisms.
- Not suitable for general-purpose computing.

Real-World Examples

- VxWorks in aerospace systems.
- QNX in automotive systems.
- Industrial robots.
- Traffic signal control systems.
- Medical monitoring devices.

Application	Why RTOS is Needed
Aircraft control systems	Flight controls must respond instantly to pilot inputs.
Air traffic control systems	Delays can compromise safety.
Medical devices (pacemakers, ventilators)	Must react within strict time limits.

Industrial robots	Precise timing is required for manufacturing tasks.
Automobile systems (ABS, airbags)	Immediate response is necessary during emergencies.
Traffic light control systems	Signals must change according to predefined timing constraints.
Missile guidance systems	Navigation calculations must occur within strict deadlines.
Nuclear power plant monitoring	Safety-critical systems require predictable responses.

4. Multiprogramming Operating System

A **Multiprogramming Operating System** is an operating system that allows **multiple programs (jobs) to reside in the main memory simultaneously** and share the CPU.

When one program is waiting for an I/O operation (such as reading a file or printing), the CPU switches to another program, thereby increasing CPU utilization.

How It Works

1. Several jobs are loaded into memory at the same time.
2. The CPU starts executing one job.
3. If that job requests an I/O operation, it is temporarily suspended.
4. The operating system immediately assigns the CPU to another ready job.
5. This process continues until all jobs are completed.

Advantages	Limitations
Higher CPU utilization	Complex OS design
Increased throughput	Memory management difficulties
Better resource utilization	CPU scheduling overhead
Reduced CPU idle time	Possibility of deadlocks
Improved system efficiency	Security and protection issues
Supports concurrent execution	Requires more memory

Batch OS	Multiprogramming OS
Executes one job at a time.	Keeps multiple jobs in memory simultaneously.
CPU may remain idle during I/O.	CPU executes another job during I/O wait.
Lower resource utilization.	Higher resource utilization.
Simpler design.	More complex design.

Distributed Operating System (DOS)

A **Distributed Operating System (DOS)** is an operating system that manages a collection of independent computers connected through a network and makes them appear to users as a **single unified computer system**.

Each computer (called a **node**) has its own processor and memory, but the distributed operating system coordinates their activities so that resources such as files, processors, and storage can be shared efficiently.

The **primary goal** of a distributed operating system is to improve **resource sharing, reliability, scalability, and performance** by allowing multiple computers to work together as one system.

Working of a Distributed Operating System

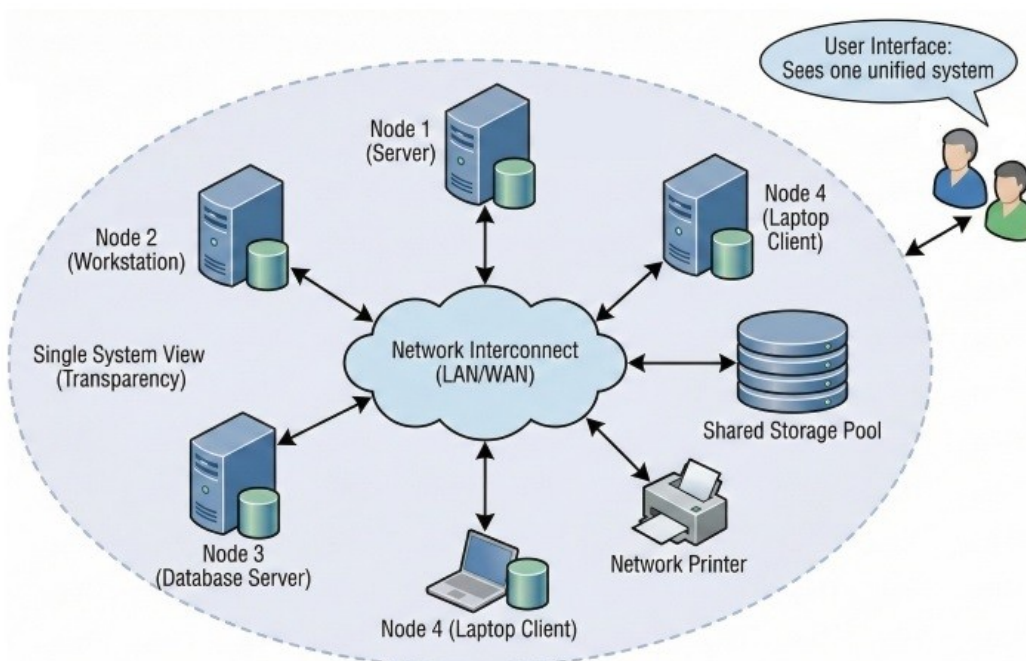
A distributed operating system consists of several computers (nodes) connected through a communication network. Each node has its own local operating system, processor, and memory. The distributed operating system coordinates these nodes so that users perceive the entire network as a single system.

The working of a distributed operating system can be explained in the following steps:

1. **User Request:** A user submits a task or program to the distributed system.
2. **Task Distribution:** The operating system analyzes the task and determines which node is best suited to execute it based on factors such as processor availability, memory, and workload.
3. **Resource Sharing:** The selected node accesses the required resources, such as files, printers, or databases, which may be located on the same node or another node in the network.
4. **Communication:** The nodes communicate with each other through the network using message passing or remote procedure calls (RPCs) to exchange data and coordinate execution.
5. **Execution:** The assigned node executes the task. If necessary, different parts of the task may be executed simultaneously on multiple nodes to improve performance.
6. **Result Collection:** After execution, the results are gathered and returned to the user as if they were processed by a single computer.

Throughout this process, the distributed operating system manages communication, synchronization, resource allocation, and fault handling to ensure smooth operation.

The operating system coordinates all the nodes so that they function as a single system.



Characteristics of a Distributed Operating System

- Multiple independent **computers work together**.
- **Resources are shared** among all nodes.
- Supports **parallel and distributed processing**.
- Provides **transparency**, making multiple computers appear as one.
- Improves system **reliability and availability**.

Advantages of a Distributed Operating System

1. Resource Sharing

Users can **share hardware resources** such as printers, storage devices, and software applications across multiple computers, **reducing overall costs**.

2. Improved Performance

Tasks can be distributed among multiple processors, allowing several computations to occur **simultaneously**. This **increases the overall processing speed**.

3. Reliability

If one computer fails, the remaining computers can continue performing tasks. This **improves system reliability and minimizes downtime**.

4. Scalability

New computers (nodes) can be added to the system without significantly affecting existing operations, making it easy to expand the system.

5. Load Balancing

The operating system **distributes workloads evenly among available nodes**, preventing any single computer from becoming overloaded.

6. Better Resource Utilization

Idle processors and memory on one node can be used to execute tasks assigned from other nodes, resulting in efficient utilization of system resources.

Drawbacks (Limitations) of a Distributed Operating System

1. Complex Design

Developing and maintaining a distributed operating system is more complex than designing a centralized operating system because it must manage communication, synchronization, and resource sharing among multiple nodes.

2. Network Dependency

The performance of the system depends heavily on the communication network. Slow or unreliable networks can reduce overall system performance.

3. Security Challenges

Since resources and data are shared across multiple computers, ensuring security and preventing unauthorized access becomes more difficult.

4. Communication Overhead

Nodes frequently exchange messages and synchronize their activities, which increases network traffic and may reduce performance.

5. Fault Detection and Recovery

Identifying and recovering from failures in distributed systems is more complicated than in single-computer systems because failures may occur at different nodes simultaneously.

6. Higher Cost

Setting up and maintaining multiple computers connected through a network requires additional hardware, networking equipment, and administrative effort.

Real-World Examples

- **Google Distributed Systems:** Google Search distributes search requests across thousands of servers.
- **Cloud Computing Platforms:** Services such as Microsoft Azure, Amazon Web Services (AWS), and Google Cloud distribute workloads across multiple servers.
- **Distributed Database Systems:** Large organizations use distributed databases where data is stored across multiple locations.
- **Scientific Computing Clusters:** Universities and research laboratories use distributed systems to perform complex scientific simulations and data analysis.

Feature	Distributed Operating System
Definition	An operating system that manages multiple networked computers as a single system.
Processing	Distributed among multiple computers (nodes).
Resource Sharing	Resources are shared across all connected nodes.
Communication	Uses a network for communication between nodes.
Advantages	Resource sharing, high performance, scalability, reliability, load balancing, better resource utilization.
Drawbacks	Complex design, network dependency, security challenges, communication overhead, difficult fault recovery, higher cost.

Comparison Between different Operating Systems

Differences between a Batch Operating System and a Real-Time Operating System

<i>Batch Operating System</i>	<i>Real-Time Operating System</i>
• Batch systems are well suited for applications that have a long execution time and do not need any quick response.	• Real-time systems are designed for applications that need quick response, thus meeting the scheduling deadlines.

• These systems do not involve any user interaction.	• These systems may require user interaction.
• FCFS scheduling algorithm is used to execute the jobs.	• Majority of real-time systems use priority-based preemptive scheduling.
• The main objective is to execute a large number of jobs as efficiently as possible.	• The main goal is to process the individual records with minimum delay.
• Time-critical device management is not needed in batch systems.	• One of the main characteristics of real-time systems is time- critical device management.

Differences between Time-Sharing Operating System and a Real- Time Operating System

<i>Time-Sharing Operating System</i>	<i>Real-Time Operating System</i>
• It is the logical extension of multiprogramming system that supports interactive users and is designed to provide a quick response time.	• This system is designed for environments where a huge number of events must be processed within fixed time constraints.
• More sophisticated memory management is needed in time-sharing systems to provide separation and protection of multiple user programs.	• In these systems, the programs remain in the main memory most of the time and there is a little swapping of programs between main and secondary memory. Thus, memory management is less demanding in real-time systems.
• Round-robin scheduling is used to execute the programs.	• Most real-time systems use priority-based pre-emptive scheduling.
• This system tends to reduce the CPU idle time by sharing it among multiple users.	• Effective resource utilization and user convenience are of secondary concern in a real- time system.

Differences between Batch and Time-Sharing Operating System

Feature	Batch Operating System	Time-Sharing Operating System
Definition	Jobs are collected and executed in batches without user interaction.	Multiple users interact with the system simultaneously by sharing CPU time.
User Interaction	No interaction during execution.	Continuous interaction is possible.
Response Time	Long response time.	Very short response time (usually seconds or less).
CPU Allocation	CPU executes one job at a time until completion or I/O wait.	CPU time is divided into small time slices among users/processes.
User Experience	Users wait until the job is completed.	Users receive immediate feedback and responses.
Resource Utilization	Better than serial processing but CPU may remain idle during I/O.	High resource utilization through rapid switching between tasks.
Turnaround Time	High (output may take minutes or hours).	Low (quick response to user requests).
Complexity	Relatively simple.	More complex due to scheduling and resource sharing.
Scheduling Method	Jobs are processed sequentially in batches.	CPU scheduling allocates time slices to multiple users/processes.
Main Objective	Maximize throughput by processing large numbers of jobs.	Provide interactive computing and fair resource sharing.
Suitable For	Repetitive, non-interactive tasks.	Interactive, multi-user environments.
Examples	Payroll processing, bill generation, exam result processing.	UNIX servers, Linux servers, online banking, cloud computing systems.

Differences between a Multiprogramming Operating System and a Time-Sharing Operating System

<i>Multiprogramming Operating System</i>	<i>Time-Sharing Operating System</i>
• It allows several programs to use the CPU at the same time and does not support user interaction.	• It is the logical extension of multiprogramming system that supports interactive users and provides a quick response time.
• The main objective is to maximize the CPU utilization by organizing programs such that the CPU has always one to execute.	• The main objective is to minimize the response time by sharing the computing resources among several users. The CPU switches between multiple users so frequently that each user gets the impression that only he or she is using the CPU alone, while actually it is the one CPU shared among many users.
• A context switch occurs only when the currently running process stalls and the CPU is to be allocated to some other process.	• A context switch occurs each time after the time slice of currently running process is over.
• Multiprogramming system is less complex than a time-sharing system.	• It is more complex than a multiprogramming system.

Differences between a Batch Operating System and a Multiprogramming Operating System

<i>Batch Operating System</i>	<i>Multiprogramming Operating System</i>
• A batch of similar jobs that consist of instructions, data, and system commands is submitted to the operator and one job is executed at a time.	• Multiple programs appear to run concurrently by rapidly switching the CPU between the programs.
• The jobs are executed in the order in which they were submitted, that is, FCFS basis.	• Multiprogramming systems need more sophisticated scheduling algorithm, as multiple processes reside in the main memory at the same time.
• These systems do not tend to achieve efficient CPU utilization.	• These systems ensure that the CPU has always something to execute, thus increasing the CPU utilization.
• Simple memory management is required.	• Some form of memory management is needed to keep several jobs in memory at the same time.
• Access to files is serial, so simple file management is needed.	• A number of processes may attempt to access a file at the same time. This demands for advanced protection and concurrency control methods.